

Anotações — Aula 3: Algoritmos de ML na Prática

Disciplina: Machine Learning — Pós-Graduação Ciência de Dados e Big Data / Estatística

Professor: Murilo Afonso Robiati Bigoto (PUC Minas)

Dataset usado: `dataset_imoveis.csv` — 500 imóveis, 9 colunas (regressão + classificação + clustering)

Notebook: `ml_imoveis.ipynb`

Mensagem da aula: “O treinamento e otimização do modelo é um ponto `.fit()`. Não precisamos enlouquecer entendendo 100% a matemática — ao longo do tempo isso virá naturalmente.”

Kaggle — Plataforma de ML

O professor mencionou o **Kaggle** ([kaggle.com](https://www.kaggle.com)) como recurso de estudo complementar.

O que é o Kaggle?

- Plataforma onde cientistas de dados resolvem problemas reais de empresas (competições)
- Repositório de datasets públicos + notebooks compartilhados
- Ótimo para ver como outras pessoas resolveram problemas similares ao seu

Como usar para aprender: Procure um dataset sobre seu tema de interesse, leia os notebooks públicos e veja como outros fizeram o pré-processamento e a modelagem. É aprendizado por osmose — você absorve boas práticas lendo código de quem já resolveu o problema.

Dataset da Aula: `dataset_imoveis.csv`

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```

from sklearn.linear_model import Lasso
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.cluster import KMeans
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import (mean_absolute_error, mean_squared_error,
                             r2_score, accuracy_score,
                             classification_report, confusion_matrix,
                             silhouette_score)

df = pd.read_csv('dataset_imoveis.csv')
print(df.shape) # (500, 9)
print(df.dtypes)

```

Colunas do dataset:

Coluna	Tipo	Descrição
regiao	str	Região do imóvel
tipo_imovel	str	Casa / Apartamento
area_m2	float	Área em m ²
quartos	float	Número de quartos
vagas_garagem	int	Vagas de garagem
idade_imovel_anos	float	Idade do imóvel
distancia_centro_km	float	Distância ao centro
preco_reais	float	TARGET regressão
faixa_preco	str	TARGET classificação (Econômico / Médio / Alto)

Definição das features para os modelos:

```

FEATURES_NUM = ['area_m2', 'quartos', 'vagas_garagem',
                 'idade_imovel_anos', 'distancia_centro_km']
FEATURES_CAT = ['regiao', 'tipo_imovel']

```

PARTE 1 — Regressão com Lasso

O que é Lasso?

Lasso (Least Absolute Shrinkage and Selection Operator) é uma **Regressão Linear com penalização L1**. Ela adiciona uma penalidade proporcional ao valor absoluto dos coeficientes, forçando alguns deles a zero — o que funciona como **seleção automática de variáveis**.

***Analogia:** Imagine que você está montando uma equipe para um projeto. A Regressão Linear normal mantém todo mundo no time. O Lasso é o gerente rígido que corta quem não contribui o suficiente — quem não agrega vai para zero, fora do modelo.*

Regularização	Penalidade	Característica
Ridge (L2)	w^2	Encolhe coeficientes, mas não zera
Lasso (L1)	w_1	Pode zerar coeficientes faz feature selection
ElasticNet	L1 + L2	Combina os dois; <code>l1_ratio</code> controla a mistura

Pré-processamento para Regressão

```
TARGET_REG = 'preco_reais'
```

```
X_reg = df[FEATURES_NUM + FEATURES_CAT]
```

```
y_reg = df[TARGET_REG]
```

```
# Split 80/20
```

```
X_train_r, X_test_r, y_train, y_test = train_test_split(
```

```
    X_reg, y_reg, test_size=0.2, random_state=42)
```

```
# Numérico: imputação por mediana + escalonamento
```

```
imputer_num = SimpleImputer(strategy='median')
```

```
scaler      = StandardScaler()
```

```
X_train_num = scaler.fit_transform(imputer_num.fit_transform(X_train_r[FEATURES_NUM
```

```
X_test_num  = scaler.transform(imputer_num.transform(X_test_r[FEATURES_NUM]))
```

```
# Categórico: imputação + One-Hot
imputer_cat = SimpleImputer(strategy='most_frequent')
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

X_train_cat = encoder.fit_transform(imputer_cat.fit_transform(X_train_r[FEATURES_CAT]))
X_test_cat = encoder.transform(imputer_cat.transform(X_test_r[FEATURES_CAT]))

# Combinar features numéricas e categóricas
import numpy as np
X_train_processed = np.hstack([X_train_num, X_train_cat])
X_test_processed = np.hstack([X_test_num, X_test_cat])
```

Atenção ao `np.hstack` vs `pd.concat` : quando as features já são arrays NumPy (saída do sklearn), use `np.hstack` . Quando são DataFrames, use `pd.concat(..., axis=1)` .

Treinamento do Lasso

```
model_lasso = Lasso(alpha=1000, random_state=42)
model_lasso.fit(X_train_processed, y_train)

y_pred_reg = model_lasso.predict(X_test_processed)
```

O que é `alpha` ? É o hiperparâmetro que controla a intensidade da penalização. Quanto maior o `alpha` , mais coeficientes serão zerados. Aprenderemos como encontrar o melhor `alpha` com técnicas de busca de hiperparâmetros.

Métricas de Regressão

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_test, y_pred_reg)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_reg))
r2 = r2_score(y_test, y_pred_reg)

print(f'MAE (Erro Médio Absoluto) : R$ {mae:,.0f}')
print(f'RMSE (Raiz do Erro Quadr.) : R$ {rmse:,.0f}')
print(f'R2 (Coef. de Determinação): {r2:.4f}')
```

```
print()
print(f' O modelo explica {r2*100:.1f}% da variância do preço')
print(f' Em média, erra R$ {mae:,.0f} por imóvel')
```

Resultado obtido no notebook:

MAE : R\$ 17.424
RMSE : R\$ 21.776
 R^2 : 0.9763 modelo explica 97,6% da variância do preço

Entendendo as Métricas

Métrica	Fórmula simplificada	O que significa
MAE	média de `	real - predito
RMSE	$\sqrt{\text{média de (real - predito)}^2}$	Penaliza erros grandes mais do que o MAE
R^2	$1 - SS_{\text{res}} / SS_{\text{tot}}$	% da variância explicada (0 a 1, quanto maior melhor)

Analogia das métricas:

- **MAE** = você compra 10 produtos e calcula o quanto errou em cada preço, tira a média. Simples.
- **RMSE** = você compra 10 produtos, mas pune erros grandes com força. Errar R\$100 num produto pesa muito mais do que errar R\$10 em 10 produtos.
- **R^2** = "qual % da bagunça nos dados meu modelo conseguiu explicar?" — 0% = não explica nada, 100% = explica tudo.

Visualizações da Regressão

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 1. Real vs Predito (ideal: pontos na diagonal)
axes[0].scatter(y_test, y_pred_reg, alpha=0.6, color='steelblue')
lims = [min(y_test.min(), y_pred_reg.min()), max(y_test.max(), y_pred_reg.max())]
axes[0].plot(lims, lims, 'r--', lw=2, label='Predição perfeita')
axes[0].set_title('Real vs Predito')
```

```
axes[0].set_xlabel('Valor Real')
axes[0].set_ylabel('Valor Predito')
axes[0].legend()

# 2. Resíduos (ideal: pontos espalhados aleatoriamente em torno do zero)
residuos = y_test - y_pred_reg
axes[1].scatter(y_pred_reg, residuos, alpha=0.6, color='teal')
axes[1].axhline(0, color='red', linestyle='--')
axes[1].set_title('Resíduos')
axes[1].set_xlabel('Valor Predito')
axes[1].set_ylabel('Resíduo (Real - Predito)')

plt.tight_layout()
plt.show()
```

Como ler o gráfico de resíduos: Se os pontos estiverem distribuídos aleatoriamente em torno da linha zero, o modelo é bom. Se houver padrões (curva, funil, tendência), o modelo tem algum problema sistemático.

PARTE 2 — Classificação com Árvore de Decisão

O que é uma Árvore de Decisão?

Uma Árvore de Decisão cria **regras hierárquicas de separação**. Em cada nó ela pergunta: “qual variável e qual limiar separa melhor meus dados em classes puras?”

***Analogia:** É como o jogo de adivinhação “Animal, Vegetal ou Mineral?”. A cada pergunta você divide o espaço de possibilidades ao meio, até chegar na resposta. A árvore faz isso automaticamente, escolhendo as melhores perguntas para minimizar a confusão entre as classes.*

Critério de divisão: Índice de Gini

O Gini mede a **impureza** de um nó. Gini = 0 significa nó completamente puro (todas as amostras da mesma classe). A árvore sempre escolhe a divisão que minimiza o Gini.

$$\text{Gini} = 1 - \sum(p^2)$$

onde p = proporção de cada classe no nó

Árvore de Decisão NÃO precisa de normalização!

A árvore não é sensível à escala das variáveis porque ela não calcula distâncias nem ajusta pesos — ela apenas testa limiares. Por isso, não é necessário aplicar `StandardScaler` para árvores.

```
# Para Árvores de Decisão: apenas imputação, sem scaling!
imputer_num_clf = SimpleImputer(strategy='median')

X_train_c_num = pd.DataFrame(
    imputer_num_clf.fit_transform(X_train_c[FEATURES_NUM]),
    columns=FEATURES_NUM, index=X_train_c.index)

X_test_c_num = pd.DataFrame(
    imputer_num_clf.transform(X_test_c[FEATURES_NUM]),
    columns=FEATURES_NUM, index=X_test_c.index)

# Categórico: imputação + One-Hot
imputer_cat_clf = SimpleImputer(strategy='most_frequent')
encoder_clf = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

X_train_c_cat = pd.DataFrame(
    encoder_clf.fit_transform(imputer_cat_clf.fit_transform(X_train_c[FEATURES_CAT])),
    columns=encoder_clf.get_feature_names_out(FEATURES_CAT), index=X_train_c.index)

X_test_c_cat = pd.DataFrame(
    encoder_clf.transform(imputer_cat_clf.transform(X_test_c[FEATURES_CAT])),
    columns=encoder_clf.get_feature_names_out(FEATURES_CAT), index=X_test_c.index)

# Combinar
X_train_c_processed = pd.concat([X_train_c_num, X_train_c_cat], axis=1)
X_test_c_processed = pd.concat([X_test_c_num, X_test_c_cat], axis=1)
```

Preparação e Split para Classificação

```
df_clf = df.dropna(subset=['faixa_preco']).copy() # remove linhas sem target

TARGET_CLF = 'faixa_preco'
X_clf = df_clf[FEATURES_NUM + FEATURES_CAT]
```

```

y_clf = df_clf[TARGET_CLF]

print(y_clf.value_counts())
# Médio: 314
# Econômico: 101
# Alto: 48 dataset desbalanceado!

# Split com stratify para manter proporção das classes no treino e teste
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_clf, y_clf, test_size=0.2, random_state=42, stratify=y_clf)

print(f'Treino: {X_train_c.shape[0]} | Teste: {X_test_c.shape[0]}')
# Treino: 370 | Teste: 93

```

Por que `stratify=y_clf` ? Garante que as proporções de cada classe (Econômico, Médio, Alto) sejam mantidas no treino e no teste. Sem isso, o conjunto de teste poderia ter muito poucos exemplos da classe “Alto” — e a avaliação ficaria distorcida.

Hiperparâmetros da Árvore

```

dt_model = DecisionTreeClassifier(
    max_depth=3,      # profundidade máxima da árvore (nível de complexidade)
    min_samples_leaf=10, # mínimo de amostras por folha (evita nós com poucos dados)
    random_state=42
)

```

Hiperparâmetro	O que controla	Efeito
<code>max_depth</code>	Profundidade máxima	Maior = mais complexo = risco de overfitting
<code>min_samples_leaf</code>	Mínimo de amostras por folha	Maior = mais simples = risco de underfitting

***Hiperparâmetros vs Parâmetros:** Os parâmetros são aprendidos pelo modelo (ex: os pesos de uma regressão). Os **hiperparâmetros** são definidos por você antes do treino e controlam o comportamento do algoritmo. Aprenderemos a encontrar os melhores hiperparâmetros automaticamente.*

Treinamento e Avaliação

```
dt_model.fit(X_train_c_processed, y_train_c)
y_pred_c = dt_model.predict(X_test_c_processed)

acc = accuracy_score(y_test_c, y_pred_c)
print(f'Acurácia: {acc:.4f} ({acc*100:.1f}%)')
print(classification_report(y_test_c, y_pred_c))
```

Matriz de Confusão

A **Matriz de Confusão** mostra quantas predições o modelo acertou e onde ele errou, para cada classe.

```
from sklearn.metrics import ConfusionMatrixDisplay

cm = confusion_matrix(y_test_c, y_pred_c, labels=dt_model.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=dt_model.classes_)
disp.plot(cmap='Blues')
plt.title('Matriz de Confusão — Árvore de Decisão')
plt.show()
```

Como ler a Matriz de Confusão:

- Diagonal principal () = acertos
- Fora da diagonal = erros

Resultado do notebook:

- “Alto”: 10 exemplos no teste modelo acertou 9
- “Econômico”: 20 exemplos modelo acertou 18
- Os 2 erros “escaparam” para outras categorias

***Analogia:** A Matriz de Confusão é o boletim escolar detalhado do modelo. A acurácia geral te diz “passou ou reprovou”. A matriz te diz “tirou 10 em Matemática, mas bombou em Português”.*

Visualizando a Árvore

```
from sklearn.tree import plot_tree

plt.figure(figsize=(18, 8))
plot_tree(
    dt_model,
    feature_names=X_train_c_processed.columns.tolist(),
    class_names=dt_model.classes_,
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title('Árvore de Decisão — 3 Níveis')
plt.show()
```

O que o professor observou na árvore:

- As variáveis mais importantes foram: **área_m2** e **distancia_centro_km**
- Elas tiveram o menor Gini (maior poder de separação das classes)
- Regra de exemplo: imóvel classificado como “Médio” = área abaixo de X m², distância acima de 15 km
- As folhas (nós finais) revelam as regras de classificação de forma transparente

PARTE 3 — Clusterização com K-Means

O que é Clusterização?

Aprendizado não-supervisionado: não há target (Y). O algoritmo descobre **grupos naturais** nos dados por conta própria.

***Analogia:** Imagine jogar centenas de fichas coloridas misturadas no chão. Você, sem saber as cores, começa a empurrar as fichas parecidas para perto umas das outras. No fim, surgem grupos naturais. Isso é o K-Means.*

Como funciona o K-Means?

1. Escolhe K centróides aleatoriamente
2. Cada ponto é atribuído ao centróide mais próximo
3. Os centróides são recalculados (nova média de cada grupo)
4. Repete até convergir

K-Means precisa de normalização, pois é baseado em distância Euclidiana.

Pré-processamento para Clusterização

```
FEATURES_CLUS_NUM = ['area_m2', 'quartos', 'vagas_garagem',
                      'idade_imovel_anos', 'distancia_centro_km', 'preco_reais']
FEATURES_CLUS_CAT = ['regiao', 'tipo_imovel']

# Numérico: imputação + padronização
imputer_clus = SimpleImputer(strategy='median')
scaler_clus = StandardScaler()

X_num_imp = imputer_clus.fit_transform(df[FEATURES_CLUS_NUM])
X_num_scl = scaler_clus.fit_transform(X_num_imp)

# Categórico: imputação + One-Hot
imputer_cat_clus = SimpleImputer(strategy='most_frequent')
encoder_clus = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

X_cat_imp = imputer_cat_clus.fit_transform(df[FEATURES_CLUS_CAT])
X_cat_enc = encoder_clus.fit_transform(X_cat_imp)

# Combinar
X_clus_prep = np.concatenate([X_num_scl, X_cat_enc], axis=1)
print(f'Shape após pré-processamento: {X_clus_prep.shape}') # (500, 13)
```

Como escolher o K ideal?

Dois métodos complementares:

- 1. Método do Cotovelo (Elbow Method)** — olha para a inércia (WCSS)
- 2. Índice de Silhueta** — mede quão bem separados os clusters estão

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

k_range = range(2, 20)
inertias = []
silhuetas = []
```

```

for k in k_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_clus_prep)
    inertias.append(km.inertia_)
    silhuetas.append(silhouette_score(X_clus_prep, labels))
    print(f' K={k:2d} | Inércia={km.inertia_:.0f} | Silhueta={silhouette_score(X_clus_prep,

# Plotar Cotovelo e Silhueta
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(list(k_range), inertias, 'o-', color='steelblue', lw=2)
axes[0].set_title('Método do Cotovelo (Elbow)')
axes[0].set_xlabel('K'); axes[0].set_ylabel('Inércia (WCSS)')

best_k = list(k_range)[np.argmax(silhuetas)]
axes[1].bar(list(k_range), silhuetas, color=['tomato' if k == best_k else 'mediumseagreen']
axes[1].set_title('Índice de Silhueta por K')
axes[1].set_xlabel('K'); axes[1].set_ylabel('Silhueta Média')

plt.tight_layout()
plt.show()

print(f'Melhor K pelo Silhueta: K={best_k}')

```

Resultado do notebook: Melhor K = 2 (Silhueta = 0.1505)

Interpretação da Silhueta

Intervalo	Qualidade dos Clusters
0.70 – 1.0	Clusters excelentes
0.50 – 0.70	Estrutura razoável
0.25 – 0.50	Estrutura fraca
< 0.25	Sem estrutura clara

Analogia do Silhueta: Imagine medir o quão “confortável” cada ponto está no seu grupo. Se um ponto está muito próximo dos seus vizinhos de cluster e muito longe dos outros clusters, ele está confortável (silhueta alta). Se está na fronteira entre dois grupos, está mal acomodado (silhueta baixa).

Treinamento e Perfil dos Clusters

```
K_FINAL = best_k
km_final = KMeans(n_clusters=K_FINAL, random_state=42, n_init=10)
cluster_labels = km_final.fit_predict(X_clus_prep)

df_clus = df.copy()
df_clus['cluster'] = cluster_labels

# Perfil médio por cluster
print("=== Perfil Médio por Cluster ===")
print(df_clus.groupby('cluster')[FEATURES_CLUS_NUM].mean().round(1))
```

Resultado do notebook (K=2):

Cluster	área_m2	quartos	dist_centro_km	preco_reais
0	92,0 m²	2,6	12,1 km	R\$ 286.347
1	149,8 m²	3,0	7,4 km	R\$ 522.152

Interpretação: **Cluster 0** = imóveis menores, mais distantes, mais baratos. **Cluster 1** = imóveis maiores, mais centrais, mais caros.

Resumo dos 3 Modelos Treinados

RESUMO DOS MODELOS TREINADOS

REGRESSÃO LASSO

MAE : R\$ 17.424

RMSE : R\$ 21.776

R² : 0.9763

CLASSIFICAÇÃO — Árvore de Decisão

Classes: Econômico | Médio | Alto

Features mais importantes: area_m2, dist_centro

CLUSTERIZAÇÃO — K-Means

K = 2

Silhueta: 0.1505 (estrutura fraca, mas detectável)

Quando usar cada tipo de modelo?

Situação	Tipo	Algoritmo da Aula
Quero prever um número	Regressão	Lasso
Quero classificar em categorias e explicar as regras	Classificação	Árvore de Decisão
Quero encontrar grupos naturais sem saber as classes	Clusterização	K-Means

Pontos-Chave para Fixar

1. **Lasso** = Regressão Linear com penalização L1 — zera coeficientes — faz feature selection automático
2. **alpha** no Lasso é um hiperparâmetro — quanto maior, mais variáveis zeradas
3. **Árvore de Decisão NÃO precisa de normalização** — ela é invariante à escala
4. **max_depth** e **min_samples_leaf** são hiperparâmetros que controlam complexidade da árvore
5. **stratify=y** no `train_test_split` é essencial quando as classes estão desbalanceadas
6. **K-Means PRECISA de normalização** — baseado em distância Euclidiana
7. O **Método do Cotovelo** e o **Índice de Silhueta** ajudam a escolher K no K-Means
8. A **Matriz de Confusão** é o “boletim completo” da classificação — vai além da acurácia
9. **Hiperparâmetros ≠ Parâmetros**: hiperparâmetros você define, parâmetros o modelo aprende

Referências

- [Scikit-learn — Lasso](#)

- [Scikit-learn — DecisionTreeClassifier](#)
- [Scikit-learn — KMeans](#)
- [Kaggle — Plataforma de datasets e competições](#)
- Géron, A. *Hands-On Machine Learning* — Caps. 4 (Linear Models), 6 (Árvores), 9 (Clustering)
- [Notebook da Aula 3 — ml_imoveis.ipynb] (material fornecido pelo professor)